通知抽取助手 · 技术文档

> 队伍：Triple Y ｜ 比赛：传智杯 AI WEB 网页开发挑战赛 版本：v1.2 ｜ 日期：2026-10-29
>
> 说明：本文由 A（李丹阳）与 C（方可圆）共同汇总；第 4.1 / 4.2 节由 A 编写；4.4 多模态由 A 与 C 共同实现；4.5 RAG 由 A 实现、C 完成接口接入。当前为省赛提交版；团队正式 Key 计划使用 DeepSeek，尚未配置，演示按 Mock + 本地 OCR 运行，真实模型记录待 Key 到位后补充。

目录

1. 项目概述
2. 需求分析
3. 系统架构设计
4. AI 技术方案（4.1 模型选型 / 4.2 Prompt 设计 / 4.3 输出稳定性 / 4.4 多模态与 OCR / 4.5 RAG 与可解释性）
5. 核心功能实现
6. 测试与部署
7. 创新点
8. 总结与展望 附录：运行命令、环境变量与测试数据

---

1. 项目概述

问题背景：校园与职场中的"办事通知"以纯文字、截图、PDF 等形式散落在群消息和公告里，关键信息（截止时间、材料清单、办理步骤、审核要求）分散在长文中。人工阅读容易漏材料、看错时间、混淆流程；通用大模型直接回答又容易编造，用户无法核验。

一句话价值：把一段通知变成**可勾选、可溯源、可提醒**的待办清单。

目标用户：在校学生（奖助学金、竞赛报名、证件办理）、社团/单位内勤（活动立项、经费报销、场地申请）、职场人士（入职材料、资质办理、行政审批）。

本期范围（P1）：文字通知解析、截图识别、文字型 PDF 提取、三档结构化输出、材料清单与缺项核对、查看依据、材料库检索与追问、清单保存与恢复。 **后续规划（P2）**：多文件合并解析、复杂条件分支、日历同步、多人协作。

2. 需求分析

2.1 典型场景

- 学生收到一条奖学金通知：需要知道**截止时间**、要交**哪些材料**、**交给谁/交到哪**、**先做什么后做什么。**
- 社团收到立项通知：需要知道**审核环节**、**依赖关系**（审核通过后才能缴费）、**风险点**（时间紧、材料复用）。
- 用户手上已有部分材料：需要系统告诉他**还缺什么**，而不是重复罗列全部要求。

2.2 功能需求

编号	需求	验收标准
F1	文字通知 → Matter JSON	返回 title / deadline / materials / step_list 等，结构符合 Schema
F2	三档复杂度自适应	simple 不出现材料/步骤；standard 有材料与步骤；complex 增加审核/依赖/预警
F3	用户情况核对	勾选材料 + 自述情况 → 已准备 / 缺少 / 需要确认 三类
F4	查看依据	每条要求可查看原文片段；无依据标"系统建议"
F5	截图解析	上传截图 → 识别文字 → Matter；格式/大小受限
F6	PDF 解析	文字型 PDF → Matter；扫描版明确提示
F7	材料库与追问	多材料上传、片段编号、检索、回答附出处、材料外拒答
F8	保存与恢复	准备清单/修改内容保存后，刷新或重启仍可恢复

2.3 非功能需求

维度	要求	实现
稳定性	模型不可用时服务不崩、前端有可读结果	视觉→本地 OCR→Mock 三层降级；统一错误信封
可解释	关键结论能追溯到原文	`source_ref.quote` + `sources` + 片段编号 + `is_suggestion`
性能	联调/演示响应可接受	Mock 实测 100 次：平均 10.4ms、P95 16.9ms；真实模型 10~30s（已在前端设 90s 超时）
安全性	密钥不入库、材料外不乱答、输出不注入	`.env` + `.gitignore`；无依据拒答；前端 `esc()` 转义
可维护性	结构清晰、可自测	9 套自测脚本 + `preflight.py` 一键自检 + `check_sync.py`

---

3. 系统架构设计

系统采用前后端分离的三层结构：

```
浏览器（前端 5173 / 静态托管）
        |  HTTP + JSON / multipart
        ▼
FastAPI 后端（8000）
  ├── 入口层：CORS、参数校验、统一响应、异常兜底
  ├── 能力层：文本抽取 / 图片解析 / PDF 提取 / 准备清单 / 查看依据 / 文件检索 / 追问
  |           / 双通知对比 / 版本与导出 / 日期核对 / 会话隔离
  ├── 抽取层：A 的 service.py（Prompt + LLM + Matter 结构 + Schema 校验）
  └── 存储层：backend/data（准备清单、文件片段索引、对比记录，JSON 原子写）
        |
        ▼
大模型（OpenAI 兼容接口，可配置）
  文本模型：LLM_MODEL        视觉模型：TEAM_LLM_MODEL
  无 Key 时：离线 Mock + 本地 OCR（RapidOCR）
```

核心设计原则：**模型只负责"读懂"，结构、判定与降级由代码兜底**，保证演示可复现、失败有反馈。

接口清单

接口	作用
`POST /api/analyze`	文字 → Matter
`POST /api/check`	用户勾选 + 自述 → 缺项核对
`POST /api/prepare-list` / `save` / `GET`	准备清单生成 / 保存 / 读取
`POST /api/evidence`、`/index`	要求 → 原文依据（无依据标为系统建议）
`POST /api/analyze-image`	截图 → Matter（视觉模型 / 本地 OCR）
`POST /api/analyze-pdf`	文字型 PDF → Matter
`POST /api/files`、`GET /api/files`、`POST /api/files/search`	材料上传、片段编号、检索
`POST /api/ask`	先检索再回答，附出处，材料外拒答
`POST /api/compare`、`GET /api/compare`、`DELETE /api/compare/{id}`	双通知对比、保留各自来源、按会话删除
`GET /api/tools/check-date`	日期核对（星期/距今/周末/常见节假日）
`GET /api/prepare-list/{id}/history`、`/export`、`DELETE`	版本历史、CSV/Markdown 导出、删除
`GET /health`	服务状态与当前模式（mock/real）

所有接口统一返回 `{success, data, error}`，且始终 HTTP 200，前端只需判断 `success`。

4.1 模型选型

4.1.1 选型约束

本项目的输入是中文办事通知，输出是字段固定的 Matter JSON，并且要同时满足“可解释、可降级、可演示”。因此模型选型不是单纯比较“谁更聪明”，而是按下面 7 个约束综合取舍：

约束	具体要求	对选型的影响
中文长文本	能稳定理解公文、通知、表格截图里的中文	优先选中文语料充分、中文指令跟随稳定的通用模型
结构化输出	能按指定 JSON 字段输出，不能夹带解释	需要支持 JSON 模式或强 JSON 指令跟随的模型
多模态	截图里可能同时有文字、表格、数字、日期	文本模型与视觉模型分开配置，视觉模型单独选型
时延与成本	单次解析 10–30 秒可接受，演示成本可控	不盲目追求最大参数模型；按任务难度选性价比档位
供应商可用性	比赛现场网络、额度、接口稳定性不可控	采用 OpenAI 兼容接口，可随时切换供应商
密钥安全	Key 不能进入前端和公开仓库	Key 只放后端 `.env`，前端只调后端
可降级	没有 Key 或接口失败时仍能演示	无 Key 走 Mock / 本地 OCR，接口失败有明确错误与兜底

4.1.2 文本模型：OpenAI 兼容接口 + 可替换供应商

文本抽取统一走 **OpenAI 兼容的 Chat Completions 接口**，通过环境变量配置：

```
# 团队正式 Key 计划使用 DeepSeek
LLM_API_KEY=<your-deepseek-key>
LLM_BASE_URL=https://api.deepseek.com/v1
LLM_MODEL=deepseek-chat
```

> 说明：开发阶段 A 曾用个人 OpenAI Key 做过接口联调，该 Key 没有提交到仓库，也不代表团队最终选型。团队正式 Key 到位后，文本模型按上面的 DeepSeek 配置重跑验收。

这样做的理由：

1. **不绑定单一供应商**：DeepSeek、通义千问、Moonshot、智谱、字节豆包、硅基流动、Ollama、vLLM 等都可以通过改 `LLM_BASE_URL` / `LLM_MODEL` 接入，比赛现场哪家额度可用就用哪家。
2. **JSON 输出成熟**：主流兼容接口都支持 `response_format={"type": "json_object"}`；即使个别供应商不支持，`_call_chat` 也会自动去掉该参数重试，避免整条链路失败。
3. **中文通知理解足够**：通用大模型对“截止时间、材料、步骤、审核、依赖”这类办事语义已经有较好的零样本能力，不需要自己标注大量数据做微调。

4. **成本可控**：抽取任务不需要复杂推理，temperature 固定为 0，按通知长度控制输入；比“每个请求都上最大模型”更适合比赛演示。
5. **可替换性**：模型名、地址、Key 全部外置，换供应商不需要改代码，也不需要重新训练。

为什么不做微调 / 不用本地小模型?

本项目的难点不是“模型知不知道什么是奖学金”，而是“能不能稳定输出指定结构、能不能不编造、能不能和代码规则配合”。微调需要标注数据、训练资源和持续维护，比赛周期内收益不稳定；本地 7B 级模型在中文长通知、JSON 稳定性、工具调用上还需要额外评测和调优。当前方案把“语义理解”交给通用模型，把“结构、判定、降级”交给代码，整体更可控，也方便随时更换供应商。

4.1.3 视觉模型：单独配置，本地 OCR 兜底

截图解析使用独立的视觉模型配置：

```
# 视觉模型单独配置；不强制与文本模型同一供应商
# 若团队 DeepSeek Key 未开通多模态，下面三项可留空，截图走本地 OCR
TEAM_LLM_API_KEY=<your-vision-key>
TEAM_LLM_BASE_URL=<vision-base-url>
TEAM_LLM_MODEL=<vision-model>
```

> 团队正式 Key 预计用于 DeepSeek 文本模型；视觉模型是否复用同一 Key，需要确认该 Key 是否支持图片输入。未确认前，截图默认走本地 RapidOCR，不影响文字主流程。

单独配置的原因：

- 视觉模型和文本模型的供应商、价格、额度可能不同，拆开后可以分别替换；
- 截图请求更贵、更慢，只在用户上传图片时调用，不拖累文本主流程；
- 视觉模型不可用时，自动降级到 **RapidOCR（onnxruntime）** 本地识别，再把文字送入统一抽取管道；
- 对“字迹模糊、表格错位、数字被切断”等视觉歧义，模型输出 `confirmation_needed`，让用户确认，而不是猜。

4.1.4 模型分工

任务	使用的模型/工具	输出	失败时
文字通知抽取	文本模型（`LLM_*`）	Matter JSON	无 Key → Mock；接口失败 → 可读错误
截图识别	视觉模型（`TEAM_LLM_*`）	Matter JSON + `confirmation_needed`	无 Key/失败 → RapidOCR → 统一抽取
PDF 文字提取	pypdf / pymupdf	纯文本 + 页码	扫描件 → 明确提示改用截图
材料库追问	文本模型 + 本地检索	回答 + 出处	无 Key → 直接返回最相关原文；无依据 → 拒答
日期核对	本地 `check_date` 工具	剩余天数/星期/风险/建议	不依赖模型，始终可执行
补充通知对比	规则差异 + 可选文本模型	`diff` + 受影响待办	模型失败 → 规则差异兜底
状态推演	文本模型 + 代码兜底	更新后的 Matter	模型失败 → `_code_level_fallback`

4.1.5 Key 配置、安全与降级

- Key 只从后端环境变量读取，前端不接触；`.env` 已被 `.gitignore` 忽略，仓库只提交 `.env.example`。
- 没有 Key 且 `LLM_ENABLE_MOCK=1` 时，`/health` 返回 `mode=mock`：文本走离线 Mock，截图走本地 OCR，保证演示不白屏。
- 配置团队 DeepSeek Key 后 `/health` 返回 `mode=real`，文本与 RAG 走真实模型；视觉模型已配置时截图走真实视觉模型，否则继续走本地 OCR。
- 开发阶段的个人 OpenAI Key 只用于本机联调，没有提交到仓库，也不作为团队验收数据。
- 真实模型接口失败时，后端返回统一错误信封，不会把异常堆栈或 Key 暴露给前端。
- 真实模式默认按 IP + 路径限流（普通接口 60 次/分钟、上传 10 次/分钟），避免演示额度被刷爆。

4.1.6 当前验收状态（重要）

截至本版提交，**团队正式 Key（计划 DeepSeek）尚未配置**，仓库默认仍按 **Mock + 本地 OCR** 运行。开发阶段 A 曾用个人 OpenAI Key 做过本机接口联调，但该 Key 未提交到仓库，也没有形成团队验收数据；真实 DeepSeek 文本模型、视觉模型、Function Calling 的 token、耗时、费用记录尚未产生，不能把个人联调或 Mock 数据当成团队真实模型效果。

项目	当前状态	团队 DeepSeek Key 到位后要补
文本解析	个人 OpenAI Key 联调过；仓库默认 Mock	用 DeepSeek 重跑 3 个样本，记录耗时、token、费用
截图识别	本地 OCR 通过；视觉模型未配置	确认 DeepSeek Key 是否支持图片输入；不支持则保持本地 OCR
RAG 追问	本地检索通过；个人 Key 联调过	用 DeepSeek 重跑回答质量、出处准确率与耗时
日期工具调用	本地工具 + 假客户端协议通过	真实 DeepSeek Function Calling 一次完整记录
补充通知对比	规则差异通过；个人 Key 联调过	用 DeepSeek 对比语义差异、误判案例与费用

> 结论：**当前按“无团队 Key / Mock 演示”提交。**团队 DeepSeek Key 到位后，按 `LLM_*`（文本）和 `TEAM_LLM_*`（视觉，可选）配置，重跑 `preflight.py` 并补充上表记录，再更新本节与 README。记录模板见 真实模型验收记录.md 。

4.2 Prompt 设计

4.2.1 Prompt 的组织方式

Prompt 不是一段“万能提示词”，而是按任务拆成 6 个专用模板，统一放在 `src/prompt.py`，并同步到 `backend/prompt.py`：

Prompt	用途	关键约束
`SYSTEM_PROMPT`	文字通知 → Matter	只抽原文、禁止编造、三档模式、字段互斥、todo 非空
`UPDATE_PROMPT`	用户操作 → 状态推演	交付动作防误勾、顺序回填、保持 ID 不变
`IMAGE_CHECK_RULES`	截图识别附加规则	只对“看不清”写 `confirmation_needed`，不把缺失当模糊
`RAG_QA_PROMPT`	材料库追问	先查材料再回答、必须附出处、无依据拒答
`COMPARE_PROMPT`	原通知 vs 补充通知	找改期/增删材料/冲突，拿不准必须让用户确认
`DATE_TOOL_SYSTEM_PROMPT`	日期工具调用	必须先调用 `check_date`，再根据工具结果给建议

拆分的好处是：每个 Prompt 只负责一类判断，规则更短、更不容易互相冲突；出问题时也能定位到具体模板，而不是在一段超长 Prompt 里猜。

4.2.2 角色与任务边界

`SYSTEM_PROMPT` 的第一条不是“尽量提取”，而是**只提取原文出现过的内容**：

- 原文明确写出的字段，`is_suggestion=false`，必须带 `source_ref.quote`；
- AI 自己补充的建议，`is_suggestion=true`，`source_ref.quote` 可以为空；
- 原文没有的字段直接省略，禁止输出 `null` 占位；
- 原文只有“12月10日前”但没有年份，按缺失处理，不编造年份；
- “各学院需组织……”这类不属于个人用户的动作，不进入个人 `step_list`。

这条边界直接对应比赛的“AI 安全与可控性”：用户能看到每条要求的原文依据，没有依据的内容会被标成“系统建议”，而不是伪装成原文。

4.2.3 系统时间注入与相对时间归一化

Prompt 里注入 `NOW = {{NOW}}`，由服务端在每次请求时写入当前时间（Asia/Shanghai）。模型负责把“下周三前”“月底前”“通知发布后 7 天内”这类相对时间换算成绝对时间，统一输出 ISO 8601（+08:00）。代码层再做一次解析和校验，避免模型输出“下周三”这种前端无法计算的值。

4.2.4 三档模式：模型先判断，代码再重算

Prompt 要求模型输出 `detected_features`、`complexity_score` 和 `mode`，但**最终模式不采信模型。**服务端在 `notification_extractor.py` 里做三件事：

1. 用固定公式重算复杂度：

```
score = 0.2*has_deadline + 0.1*has_multiple_reminders + 0.2*has_multiple_steps
      + 0.2*has_materials + 0.1*has_review + 0.1*has_dependencies
      + 0.1*has_recurrence + 0.1*has_sub_steps
```

2. 按优先级判定：

```
has_sub_steps / has_review / has_dependencies → complex
score < 0.3 → simple
0.3 ≤ score ≤ 0.6 → standard
score > 0.6 → complex
```

3. 取“模型给的 mode”和“公式算出的 mode”中**更高**的一档，防止模型把复杂通知降级成 simple，导致材料/步骤被裁掉。

这样同一份通知在不同模型、不同温度下，模式判定仍然稳定、可复现。

4.2.5 字段互斥与 `enabled_sections`

三档模式不是“字段多一点少一点”，而是有明确的互斥表。`enforce_mode` 会直接删除当前模式不允许的字段：

模式	允许的 `enabled_sections`	主要保留字段	明确禁止
simple	summary, schedule, source	title, deadline, reminders, sources	materials, step_list, todo_list, warning_list, attachments, location 等
standard	summary, schedule, content, steps, materials, source	加 materials, step_list, attachments, key_points	recurrence, permissions, audit_log, comments
complex	全部 8 个区块	全部字段	无

另外，simple/standard 的步骤里会删除 `sub_step_list`、`dependencies`、`parallel`、`condition`、`review`、`owner` 等复杂字段；standard 只保留 `deadline_risk`、`missing_material` 两类预警。这样前端只渲染 `enabled_sections` 声明的区块，不会出现“简单通知给一堆空模块”。

4.2.6 ID、命名与依赖规则

- `step_id` / `material_id` / `todo_id` 必须按顺序编号，例如 `step_1`、`mat_1`、`todo_1`；
- `material_name` 必须是 2–10 字的简短名词短语，例如“报名表”“成绩单”，不能把整句话塞进去；
- `step_list` 是“阶段”（2–5 个），`todo_list` 是“阶段里的具体动作”，两者粒度必须分开；
- 只有原文明确写了“A 完成后才能做 B”，才写 `dependencies`；没有明确依赖就按并行处理；
- `todo_list` 绝对不允许为空，`action` 必须是完整动宾短语，例如“准备并填写报名表”。

4.2.7 结构化输出的稳定性

Prompt 层和代码层共同保证 JSON 稳定：

1. **Prompt 层**：只输出 JSON、不要 Markdown、不要解释；字段名和类型写死；空数组/空对象省略；`temperature=0`。
2. **接口层**：优先使用 `response_format={"type": "json_object"}`；供应商不支持时自动去掉该参数重试。
3. **解析层**：去掉 ``` ```json 代码围栏后再 json.loads`；解析失败返回可读错误，不把脏数据交给前端。
4. **校验层**：`jsonschema` 校验 Matter 结构，失败直接报错。
5. **修补层**：`_fix_todo_actions` 补空 action，`_fix_step_titles` 补空 title，`_rebuild_missing_items` 重建缺项，`_merge_ai_output` 以旧结构为基准合并，模型漏字段不会把原数据冲掉。
6. **兜底层**：模型失败时走 `_code_level_fallback`，至少保留基础状态联动。

4.2.8 状态推演 Prompt：防止“材料齐了就把提交动作也勾了”

`UPDATE_PROMPT` 的核心不是“让模型改状态”，而是给状态推演立规矩：

- **交付动作防误勾**：包含“提交/发送/上交/上传/打包/初审/审核/盖章”的动作，即使材料都准备好了，也不能自动标完成，必须用户明确确认；
- **顺序与并行区分**：有明确“先 A 后 B”的依赖，完成 B 时沿链回填 A；并行分支互不影响；
- **ID 不变**：禁止重命名或删除业务条目；
- **只改被提到的材料**：用户没说的材料保持原状，不猜；
- **代码兜底**：模型输出漂移时，`_sequential_backfill`、`_code_level_fallback`、`_fix_todo_actions` 继续完成基础联动。

4.2.9 图片 Prompt：只对“看不清”要求确认

`IMAGE_CHECK_RULES` 明确区分两类情况：

- **视觉歧义**（字迹模糊、纸张皱褶、遮挡、表格错位、数字被切断）→ 写入 `confirmation_needed`；
- **原文就没写**（只有“12月10日前”没有年份、单位没写全称、材料压根没提）→ 属于缺失，不写 `confirmation_needed`，也不编造。

这条规则避免把“信息缺失”误报成“识别失败”，也避免模型用确认弹窗掩盖自己没抽到的事实。

4.2.10 RAG / 对比 / 工具调用 Prompt

- **RAG 追问**：先检索、再回答；回答必须附“文件名 - 片段ID - 页码”；检索不到就输出“【未找到依据】……”，不允许用外部常识补答。
- **补充通知对比**：明确写“以此为准/原通知作废”时，改期按补充通知处理；没写时，改期、删材料一律要求用户确认；只输出差异，不重复编造原文。
- **日期工具调用**：要求模型先调用 `check_date`，再根据工具返回的 `days_left / is_weekend / suggested_finish_at` 生成建议；模型不调用时，代码层仍会执行一次工具，保证“至少有一次真实调用”。
- **状态更新**：只允许改状态和补缺项，不允许改业务字段；模型失败时保留原清单并返回错误。

4.2.11 Prompt 迭代经验

这版 Prompt 不是一次写成的，主要迭代点：

1. 从“让模型自由总结”改成“固定字段 + JSON Schema”，解决前端无法渲染的问题；
2. 从“模型自己给 mode”改成“公式重算 + 取更高档”，解决复杂通知被降级；
3. 从“材料/步骤混在一起”改成“step 是阶段、todo 是动作”，解决清单粒度混乱；
4. 从“没有依据也硬答”改成“source_ref + is_suggestion + 无依据拒答”，解决幻觉；
5. 从“改期直接覆盖”改成“差异预览 + 用户确认 + change_log”，解决补充通知误覆盖；

6. 从“日期让模型算”改成“模型调工具、代码执行”，解决模型算错日期和周末的问题。

4.3 输出稳定性（C 侧实现）

1. **复杂度重算**：不采信模型给的分数，用固定公式 `0.2 • deadline + 0.1 • 多提醒 + 0.2 • 多步骤 + 0.2 • 材料 + 0.1 • 审核 + 0.1 • 依赖 + 0.1 • 周期 + 0.1 • 子步骤`， 再按 `has_sub_steps/has_review/has_dependencies → complex`、`<0.3 simple`、`0.3 – 0.6 standard`、`>0.6 complex` 判定。
2. **字段裁剪**：`enforce_mode` 按模式删除互斥字段（simple 不出现 materials/step_list/todo_list 等），保证前端渲染契约稳定。
3. **Schema 校验**：`jsonschema` 校验 Matter 结构，失败直接返回可读错误，不把脏数据交给前端。
4. **三层降级**：真实模型 → 本地 OCR（图片）→ 离线 Mock（无 Key），任一层可用即可演示。

4.4 多模态与 OCR（A + C 共同实现）

- **分工**：A 侧负责截图视觉抽取函数 `analyze_notification_from_image`、截图专用 Prompt 规则（表格/人数/日期/材料/金额）与 `confirmation_needed` 判定；C 侧负责 `/api/analyze-image` 接口、文件大小/格式校验、本地 RapidOCR 兜底与统一抽取管道。
- **截图**：`POST /api/analyze-image`，multipart 字段 `image`，JPG/PNG/WEBP，≤5MB。 有团队 Key 时调用 `analyze_notification_from_image`（多模态 + 截图专用规则：表格/人数/日期/材料/金额，模糊写入 `confirmation_needed`）； 无 Key 或视觉调用失败时，自动用 **RapidOCR（onnxruntime）** 本地识别，再走统一抽取管道。
- **PDF**：`POST /api/analyze-pdf`，字段 `file`，≤10MB、≤30页，仅文字型；用 pypdf 提取文字。 扫描版提取不到文字时返回"请改用清晰截图或直接粘贴文字"，避免空结果。
- 统一限制端点：`/api/analyze-image/limits`、`/api/analyze-pdf/limits`，前端提示与后端完全一致。

4.5 RAG 与可解释性（A 实现，C 接入）

- **分工**：A 侧实现 `NotificationRetriever` 检索器、中文二元组/LCS 打分、`ask_question_with_rag` 先检索后回答、`source_ref` / `is_suggestion` 溯源规则；C 侧负责材料上传与存储、`/api/files`、`/api/ask`、`/api/evidence` 接口接入。
- **片段编号**：上传材料按 400 字 / 80 字重叠切分，生成 `file_xxx_c1`、`_c2`…，记录 `char_start/char_end`。
- **检索**：`POST /api/files/search` 对片段做中文二元组重叠打分（命中片段 + 精确子串加权），返回文件名、片段号、原文、分数。
- **追问**：`POST /api/ask` 先检索；有 Key 时把检索片段作为上下文交给 `ask_question`，无 Key 时直接返回最相关原文与出处。

- **不敢答**：检索为空时返回"在已上传的材料中没有找到...无法回答"，不调用模型编造。
- **依据**：`POST /api/evidence` 对材料/步骤/待办/附件/截止/预警/要点返回 `source_ref.quote`；无引用的条目标 `is_suggestion=true`，前端以黄色"系统建议"展示。
- **双通知对比**：`POST /api/compare` 分别解析并保留两份通知的文件名、原文和 Matter，对比截止时间、材料与步骤差异；记录支持按会话读取和删除。

5. 核心功能实现（后端）

- **统一入口**：`main.py` 负责 CORS、参数校验（空文本 / 超长 10000 字）、multipart 大小与格式校验、统一异常兜底。
- **统一管道**：`analysis_pipeline.analyze_with_fallback` 负责"真实模型 / 离线 Mock"切换，文本、PDF、图片三条入口共用。
- **存储**：准备清单（`data/prepare_lists.json`）与文件索引（`data/files.json`）采用线程锁 + 临时文件原子写，容器通过 volume 持久化。
- **状态联动**：勾选材料 → 关联待办自动 `completed`；依赖全部完成 → `blocked` 解除为 `pending`；状态枚举兼容 `pending/completed/blocked/need_confirm`。
- **版本与导出**：准备清单每次确认保存后 `version` 递增，保留 `previous_todo_list` 和最近 5 版 `history`；支持服务端导出 CSV（UTF-8 BOM）与 Markdown。
- **日期核对**：`GET /api/tools/check-date` 返回星期、距今、周末、常见法定节假日与“是否工作日”，并提示调休需人工确认。
- **会话隔离与删除**：前端通过 `X-Session-Id` 区分浏览器会话，文件库、准备清单、通知对比互不可见；三类数据均提供删除接口。
- **限额与密钥安全**：真实模型模式默认按 IP + 路径限流（普通接口 60 次/分钟、上传 10 次/分钟），Key 只从环境变量读取，不进入仓库。

6. 测试与部署

自动化测试（9 套 + 一键自检，全部通过）

脚本	项数	覆盖
`smoke_test.py`	10	健康检查、正常/空/超长文本、非法 JSON、CORS、Mock 性能
`smoke_test_check.py`	10	缺项差集、全勾选、自述冲突、参数兜底
`smoke_test_prepare.py`	11	生成、保存、读取、修改内容持久化
`smoke_test_evidence.py`	8	索引、单条依据、无依据 → 建议、参数校验
`smoke_test_image.py`	6	limits、合法图、格式/大小/空文件
`smoke_test_pdf.py`	6	文字型 PDF、扫描件、页数/大小限制
`smoke_test_files.py`	10	上传、片段编号、检索、追问、材料外拒答
`smoke_test_acceptance.py`	3 样本	三档模式端到端验收
`smoke_test_hardening.py`	16	双通知、版本历史、导出、日期、会话隔离、删除
`preflight.py`	13	演示前一键自检（含 10/28–10/30 新链路）

部署

- Docker：`backend/Dockerfile`（含 pypdf、RapidOCR）+ `docker-compose.yml`，`backend/data` volume 持久化，内置 HEALTHCHECK。
- Nginx 反代 + HTTPS，`proxy_read_timeout 180s`、`client_max_body_size 12m`。
- 详见 `docs/部署说明.md`。

---

7. 创新点

7.1 技术创新

1. **公式化复杂度判定 + 字段裁剪**：不让模型"凭感觉"给分，服务端用固定公式重算 complexity_score，并按模式强制裁剪字段，保证同一份通知的输出稳定、可复现。
2. **三档结构自适应**：simple / standard / complex 按需输出，前端只渲染 `enabled_sections` 中声明的区块，避免"简单通知给一堆空模块"。

3. **全链路降级**：视觉大模型不可用 → 本地 OCR（RapidOCR）→ 离线 Mock；文本模型不可用 → 离线抽取。演示与前端联调不会因为 Key/网络中断而白屏。
4. **可解释与防幻觉**：字段级 `source_ref` + "查看依据"接口；无原文依据的条目标为"系统建议"；材料库检索不到时直接回答"未找到依据"，不编造。

7.2 应用创新

1. **从"读通知"到"能执行"**：输出不是一段摘要，而是可勾选的材料清单、可排序的步骤、可提醒的 DDL。
2. **用户情况驱动的缺项核对**：把"通知要求"和"用户已有"做差集，直接告诉用户还缺什么、什么需要确认。
3. **材料库 + 追问**：上传多份材料后可以问"视频有什么要求"，答案附文件名与片段编号，形成"办事资料助手"。

8. 总结与展望

已完成：文字/截图/PDF 三种输入通道；三档结构化输出；材料清单与缺项核对；查看依据；材料库片段检索与追问；准备清单保存与恢复；双通知对比与变更确认；清单版本历史与 CSV/Markdown 导出；日期核对工具；会话隔离与删除；9 套自动化测试与一键自检；Docker 部署方案与演示素材。

局限：团队正式 Key（计划 DeepSeek）尚未配置，当前文本走 Mock、截图走本地 OCR；开发阶段 A 使用个人 OpenAI Key 做过联调，但不作为团队验收数据；扫描版 PDF 依赖截图通道；复杂表格/手写体识别精度取决于模型；RAG 采用词法/二元组检索而非向量检索，同义改写可能漏召；补充通知对比的语义差异仍可能漏判；会话隔离基于 `X-Session-Id`，不是登录鉴权；节假日只覆盖常见固定假期，调休需人工确认。

展望：多文件合并解析、复杂条件分支（"若为应届生需额外提交…"）、向量检索与重排、日历/飞书同步、多人协作清单。

附录

A.1 运行命令

```
cd backend
.\.venv\Scripts\python.exe -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
cd ..\frontend
npm run dev

# 重新生成技术文档 HTML + PDF（A 修改 4.1/4.2 后运行）
powershell -ExecutionPolicy Bypass -File ..\scripts\build_tech_pdf.ps1
```

A.2 环境变量

变量	说明
`LLM_ENABLE_MOCK`	无 Key 时是否走离线 Mock
`LLM_API_KEY` / `LLM_BASE_URL` / `LLM_MODEL`	文本模型
`TEAM_LLM_API_KEY` / `TEAM_LLM_BASE_URL` / `TEAM_LLM_MODEL`	视觉模型（截图）
`MAX_TEXT_LENGTH`	文本长度上限（默认 3000，演示配 10000）
`RATE_LIMIT_PER_MINUTE`	普通模型接口每分钟限额（默认 60）
`UPLOAD_RATE_LIMIT_PER_MINUTE`	上传接口每分钟限额（默认 10）
`RATE_LIMIT_ENFORCE_IN_MOCK`	Mock 模式是否强制限流（默认 0）
`X-Session-Id`	前端请求头，用于用户数据隔离（不是登录鉴权）

A.3 测试结果

脚本	结果
9 套自测（接口/缺项/清单/依据/图片/PDF/文件/追问/加固）	全部通过
`smoke_test_acceptance.py`	simple / standard / complex 3/3
`preflight.py`	13/13，可开始演示
`smoke_test_hardening.py`	16/16
`smoke_test_perf.py`（Mock 100 次）	平均约 10–14ms，P95 约 17ms
`docs/技术文档.pdf`	已重新导出 16 页最终版（≤30 页），含 4.1/4.2